

Apprentissage non supervisé M1 OIVM — UPEC

TP / Devoir : Classification hiérarchique
Durée indicative : 4h — Python / Jupyter Notebook

Année 2025/2026

Objectifs : — Comprendre le principe de la classification hiérarchique (agglomérative). — Manipuler différentes méthodes d'agrégation (Ward, complete, average). — Étudier l'impact du choix de la distance (euclidienne, Manhattan). — Comparer classification hiérarchique et K-means (avec ou sans ACP).

Jeux de données : — Iris de `sklearn.datasets` (3 classes, 4 variables). — Wine de `sklearn.datasets` (3 cépages, 13 variables).

Prérequis : Bases de K-means, prétraitement (centrage-réduction), ACP (TP précédents).

Livrables : Notebook commenté + figures (dendrogrammes, partitions) + synthèse finale.

Exercice 1 — Rappels sur la classification hiérarchique (30 min)

- Expliquez la différence entre :
 - classification **hiérarchique agglomérative** (bottom-up)
 - classification **divisive** (top-down)
- Décrivez le principe général d'un algorithme de classification hiérarchique agglomérative :
 - état initial,
 - fusion des clusters,
 - critère utilisé pour décider de la fusion,
 - production du dendrogramme.
- Citez plusieurs méthodes d'agrégation (ou *linkage*) et expliquez brièvement :
 - **single, complete, average, Ward.**

Question 1.1.1 : Quels avantages présente la classification hiérarchique par rapport à K-means ?

Question 1.1.2 : Quelles sont ses limitations principales (complexité, sensibilité au choix de la distance, etc.) ?

Exercice 2 — Première classification hiérarchique sur Iris (60 min)

On commence par le jeu de données **Iris**, plus simple à visualiser (4 variables, 3 classes).

— Chargez le jeu de données **Iris**, séparez les variables X et les étiquettes réelles y . — Standardisez éventuellement les variables (à discuter).

```
1 # from sklearn.datasets import load_iris
2 # from sklearn.preprocessing import StandardScaler
3 # import pandas as pd
4 # import numpy as np
5 #
6 # # 1) charger Iris
7 # # completer
8 #
9 # # 2) construire X (DataFrame) et y (Series)
```

```

10 # # completer
11 #
12 # # 3) (optionnel) standardiser X -> X_scaled
13 # # completer

```

On souhaite maintenant effectuer une classification hiérarchique agglomérative et tracer un **dendrogramme**.

- Utilisez le module `scipy.cluster.hierarchy` (fonctions `linkage` et `dendrogram`) pour :
- calculer une matrice de liaison (par ex. `method='ward'`),
- afficher le dendrogramme.

```

1 # from scipy.cluster.hierarchy import linkage, dendrogram
2 # import matplotlib.pyplot as plt
3 #
4 # # Z = linkage(..., method="ward", metric="euclidean")
5 # # completer
6 #
7 # # plt.figure(...)
8 # # dendrogram(Z, ...)
9 # # plt.title("Dendrogramme Iris - Ward")
10 # # plt.xlabel("Observations"); plt.ylabel("Distance")
11 # # plt.show()

```

— À partir du dendrogramme, choisissez une hauteur de coupure pour obtenir 3 clusters. Utilisez `fcluster` pour récupérer les étiquettes de clusters.

```

1 # from scipy.cluster.hierarchy import fcluster
2 #
3 # # labels_hc = fcluster(Z, t=..., criterion="maxclust")
4 # # completer

```

Question 2.1.1 : Le dendrogramme suggère-t-il une structure clairement en 3 groupes ?

Question 2.1.2 : Comparez les clusters trouvés aux vraies classes d'Iris (tableau croisé, taux d'accord).

```

1 # import pandas as pd
2 #
3 # # tableau crois clusters vs classes r elles
4 # # completer

```

Exercice 3 — Effet des méthodes d'agrégation (Iris) (45 min)

On étudie maintenant l'impact de la méthode d'agrégation (*linkage*) sur le dendrogramme et la partition.

- Répétez l'Exercice 2 avec différentes méthodes :
- `method='single'`,
- `method='complete'`,
- `method='average'`,
- `method='ward'` (si compatible avec la distance choisie).
- Pour chaque méthode :
- a) tracez le dendrogramme,
- b) choisissez un découpage en 3 clusters,
- c) construisez le tableau croisé clusters vs classes réelles,

d) calculez un indicateur de qualité (par ex. ARI).

```
1 # from sklearn.metrics import adjusted_rand_score
2 #
3 # methods = ["single", "complete", "average", "ward"]
4 # results = []
5 # for m in methods:
6 #     # 1) calculer Z = linkage(..., method=m, metric="euclidean")
7 #     # completer
8 #
9 #     # 2) labels_hc = fcluster(..., t=3, criterion="maxclust")
10 #    # completer
11 #
12 #    # 3) calculer ARI = adjusted_rand_score(y, labels_hc)
13 #    # completer
14 #
15 #    # 4) stocker ARI et/ou d'autres informations dans results
16 #    pass
17 #
18 # # afficher results
19 # # completer
```

Question 3.1.1 : Quelles méthodes d'agrégation donnent les partitions les plus cohérentes avec les classes ?

Question 3.1.2 : Que remarquez-vous sur la forme des dendrogrammes selon `single`, `complete`, `average`, `ward` ?

Exercice 4 — Distances, énergie et jeu de données Wine (60 min)

On étudie maintenant le jeu de données Wine, plus dimensionnel (13 variables).

— Chargez le dataset Wine, standardisez les variables et stockez les étiquettes réelles (3 cépages).

```
1 # from sklearn.datasets import load_wine
2 # from sklearn.preprocessing import StandardScaler
3 #
4 # # 1) charger Wine
5 # # completer
6 #
7 # # 2) construire X_wine (DataFrame) et y_wine (Series)
8 # # completer
9 #
10 # # 3) standardiser X_wine -> Xw_scaled
11 # # completer
```

On veut comparer différents couples (distance, linkage) en termes de qualité de partition :

- distance euclidienne avec `complete`, `average`, `ward`,
- distance Manhattan (cityblock) avec `complete`, `average`.

Pour chaque combinaison distance / linkage :

- a) effectuez la classification hiérarchique (3 clusters),
- b) calculez l'ARI et la silhouette moyenne,
- c) commentez les résultats.

```
1 # from scipy.cluster.hierarchy import linkage, fcluster
2 # from sklearn.metrics import adjusted_rand_score, silhouette_score
```

```

3 #
4 # configs = [
5 #     ("euclidean", "complete"),
6 #     ("euclidean", "average"),
7 #     ("euclidean", "ward"),
8 #     ("cityblock", "complete"),
9 #     ("cityblock", "average"),
10 # ]
11 #
12 # results_wine = []
13 # for dist, meth in configs:
14 #     # 1) Z = linkage(Xw_scaled, method=meth, metric=dist)
15 #     #     attention: Ward n cessite la distance euclidienne
16 #     # completer
17 #
18 #     # 2) labels_hc = fcluster(Z, t=3, criterion="maxclust")
19 #     # completer
20 #
21 #     # 3) ARI = adjusted_rand_score(y_wine, labels_hc)
22 #     #     sil = silhouette_score(Xw_scaled, labels_hc)
23 #     # completer
24 #
25 #     #4) stocker (dist, meth, ARI, sil) dans results_wine
26 #     pass
27 #
28 # # afficher results_wine
29 # # completer

```

Question 4.1.1 : Quels couples (distance, méthode d'agrégation) semblent les plus adaptés au jeu Wine ?

Question 4.1.2 : Observez-vous un effet marqué du choix de la distance (euclidienne vs Manhattan) ?

Exercice 5 — Lien avec K-means et ACP (45 min)

On compare maintenant la classification hiérarchique au K-means (TP précédent), avec ou sans réduction de dimension par ACP.

— Appliquez un K-means avec $k = 3$ sur `Xw_scaled` (même paramétrage que dans le TP K-means).

— Calculez les mêmes indicateurs que précédemment (ARI, silhouette) et comparez avec la meilleure configuration hiérarchique obtenue à l'Exercice 4.

```

1 # from sklearn.cluster import KMeans
2 #
3 # # K-means sur Xw_scaled avec k=3
4 # # completer
5 #
6 # # calculer ARI et silhouette pour K-means
7 # # completer

```

— Réalisez une ACP sur `Xw_scaled` (par exemple en conservant les 2 ou 3 premières composantes expliquant 80–90% de la variance), puis refaites :

- une classification hiérarchique (configuration choisie),
- un K-means,

sur les données projetées (scores ACP).

```

1 # from sklearn.decomposition import PCA
2 #
3 # # pca = PCA(n_components=...)
4 # # Xw_pca = pca.fit_transform(Xw_scaled)
5 # # completer
6 #
7 # # classification hi rarchique sur Xw_pca (meilleure config)
8 # # completer
9 #
10 # # K-means sur Xw_pca
11 # # completer
12 #
13 # # comparer ARI / silhouette avant/apr s ACP
14 # # completer

```

Question 5.1.1 : L'ACP améliore-t-elle la qualité du clustering (hiérarchique et/ou K-means) ?

Question 5.1.2 : Dans ce contexte, laquelle des deux approches (K-means vs classification hiérarchique) vous semble la plus adaptée ?

Exercice 6 — Synthèse (15 min)

- Rédigez une synthèse (10–15 lignes) répondant aux points suivants :
- intérêts de la classification hiérarchique par rapport à K-means,
- rôle du choix de la distance et de la méthode d'agrégation,
- cas où l'on privilégiera K-means, cas où l'on privilégiera une méthode hiérarchique,
- rôle possible d'une étape de réduction de dimension (ACP) avant le clustering.